مـدن الـمـهـن و الـكـفـاءات
Cités des métiers et des compétences

OFPPT
La voie de l'avenir

مكتب التكوين المهني و إنعاش الشغل
Office de la Formation Professionnelle
et de la Promotion du Travail

Module: Développer en back-end

Protection CSRF

Filière: Développement digital – option web full stack

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(**C**ross-**S**ite **R**equest **F**orgery)

Falsification de requêtes intersites

✓Présentation

Les falsifications de requêtes intersites sont un type d'exploit malveillant par lequel des commandes non autorisées sont exécutées au nom d'un utilisateur authentifié.

Il s'agit d'un type d'attaque en ligne dans lequel l'attaquant envoie des demandes en tant qu'utilisateur autorisé à un système en obtenant des informations d'accès d'un utilisateur particulier de ce système et effectue différents types d'activités malveillantes en utilisant l'identité de cet utilisateur. L'impact de cette attaque dépend des privilèges de la victime sur le système.

Si la victime est un utilisateur normal, cela n'affectera que les données personnelles de la victime. Mais si la victime est l'administrateur du système, l'attaquant peut endommager l'ensemble du système.

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓Fonctionnement du CSRF

- La falsification de requêtes intersites (CSRF) est un type d'attaque effectué par l'attaquant pour envoyer des requêtes à un système avec l'aide d'un utilisateur autorisé auquel le système fait confiance.
- Laravel fournit une protection contre les attaques CSRF en générant un jeton CSRF . Ce jeton CSRF est généré automatiquement pour chaque utilisateur. Ce jeton n'est rien d'autre qu'une chaîne aléatoire gérée par l'application Laravel pour vérifier les demandes des utilisateurs.
- Cette protection de jeton CSRF peut être appliquée à n'importe quel formulaire HTML dans l'application Laravel en spécifiant un champ de formulaire caché du jeton CSRF.

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓Fonctionnement du CSRF(suite)

- Une des manières efficaces de se prémunir contre les attaques CSRF est d'accompagner chaque formulaire à protéger d'un jeton (en anglais "token") d'identification de session.
- Pour ce faire, lorsqu'un visiteur se connecte sur votre site, vous enregistrez en session une chaîne de caractère aléatoire.
- Cette même chaîne est récupérée en session et passée au formulaire, sous la forme d'un champ de type "hidden".
- Lorsque le formulaire est posté, votre système de validation doit vérifier que le champ caché contient la même valeur que celle enregistrée en session.
- Si ça n'est pas le cas, ça signifie que quelqu'un a essayé de soumettre le formulaire depuis un autre endroit que votre site. Il faut alors annuler le traitement de ce formulaire.

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓Fonctionnement du CSRF(fin)

Laravel inclut un plug-in CSRF intégré, qui génère des jetons pour chaque session utilisateur active. Ces jetons vérifient que les opérations ou requêtes sont envoyées par l'utilisateur authentifié concerné.

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓Empêcher les requêtes CSRF

- CSRF est implémenté dans les formulaires HTML déclarés dans les applications Web. Vous devez inclure un jeton CSRF validé masqué dans le formulaire, afin que le middleware de protection CSRF de Laravel puisse valider la demande.
- La protection CSRF peut être implémentée dans Laravel à l'aide de n'importe quel formulaire HTML avec une forme cachée de jeton CSRF et la demande de l'utilisateur est validée à l'aide du middleware CSRF VerifyCsrfToken.

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ Empêcher les requêtes CSRF (@csrf)

- L'une des options suivantes peut être utilisée pour générer un jeton CSRF.

@csrf

- ❑ C'est une directive de Blade pour générer un champ de jeton qui sera utilisé pour la vérification. Il génère un champ de saisie masqué.
- ❑ Lorsque la directive blade @csrf est utilisée, une balise **input** de type='**hidden**' est ajoutée au formulaire.
- ❑ La valeur sera le jeton CSRF qui sera envoyé dans le cadre des données du formulaire lorsque le formulaire est soumis.

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ Empêcher les requêtes CSRF(@csrf)

Syntaxe

```
[...]
    <form action="{{ url('/.....')  }}" method="post">
       @csrf
      [...]
    </form>
[...]
```

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓Empêcher les requêtes CSRF(@csrf)

Activité

- Créez un fichier de **vue** Laravel nommé **form-csrf.blade.php** avec le code HTML suivant où la directive @csrf est utilisée pour générer le jeton CSRF.

Développer en back-end

```
<div class="container mt-5">
    <form action="" method="post">
      @csrf
     <div class="row">
        <div class="form-group col-6 lg" >
            <label>Prénom & Nom</label>
            <input type="text" class="form-control" name="prenomNom"
id="prenomNom">
        </div>
        <div class="form-group col-6 lg">
         <label>Email</label>
         <input type="email" class="form-control" name="email"
            id="email">
        </div>
      </div>
```

Développer en back-end

```
<div class="row">
        <div class="form-group col-12 lg">
            <label>Message</label>
            <textarea class="form-control" name="message"
id="message"
                rows="4"></textarea>
        </div>
</div>
        <input type="submit" name="send" value="Submit"
class="btn btn-primary btn-block">
      </form>
   </div>
```

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ Empêcher les requêtes CSRF(@csrf)

Ajoutez la route suivante dans le fichier **web.php** pour charger le fichier de vue dans le navigateur. Lorsque l'utilisateur donnera **form-csrf** après l'URL de base (***http://127.0.0.1:8000/form-csrf***), il recherchera le fichier ***form-csrf.blade.php*** dans le dossier de vue du projet Laravel.

```
Route::get('/form-csrf', function () {
        return view('form-csrf');
});
```

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ Empêcher les requêtes CSRF(fin)

Si vous inspectez la page après l'exécution, vous obtiendrez la sortie comme ci-dessous. Ici, un champ masqué avec la valeur est généré automatiquement par la directive **@csrf** .

```
▼<div class="container mt-5">
  ▼<form action>
      <input type="hidden" name="_token" value="3bv3JkklBnsafdLNyxUe0gXCreX99ztI12Of5QNB"> == $0
    ▶<div class="row">…</div>
    ▶<div class="row">…</div>
      <input type="submit" name="send" value="Submit" class="btn btn-primary btn-block">
    </form>
  </div>
```

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ Empêcher les requêtes CSRF(csrf_token ())

csrf_token ()

Cette fonction peut être utilisée dans la balise meta et le champ de saisie masqué du formulaire HTML. Il génère une chaîne aléatoire en tant que jeton CSRF.

Créez un fichier de vue Laravel nommé csrf2.blade.php avec le code HTML suivant où la fonction csrf_token () est utilisée pour générer le jeton CSRF. Cette fonction est utilisée comme valeur de l'attribut value du champ masqué et est utilisée avec deux accolades.

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ Empêcher les requêtes CSRF(csrf_token ())

✓ **Syntaxe**

```
<form method = "POST">
  <input type = "hidden" name = "_ token" value = "{{csrf_token ()}}">
  .....
  .....
</form>
```

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ Empêcher les requêtes CSRF(csrf_token ())

✓ **Activité**

Créez un fichier de vue Laravel nommé *form-csrf_token.blade.php* avec le code HTML suivant où la fonction **csrf_token ()** est utilisée pour générer le jeton CSRF.

Cette fonction est utilisée comme valeur de l'attribut value du champ masqué et est utilisée avec deux accolades.

Développer en back-end

```
<div class="container mt-5">
        <form action="" method="post">

<div class="row">
<input type="hidden" name="_token" value="{{ csrf_token() }}">
            <div class="form-group col-6 lg" >
                <label>Prénom & Nom</label>
                <input type="text" class="form-control" name="prenomNom"
id="prenomNom">
            </div>

            <div class="form-group col-6 lg">
                <label>Email</label>
                <input type="email" class="form-control" name="email"
                    id="email">
            </div>
</div>
```

Développer en back-end

```
<div class="row">
        <div class="form-group col-12 lg">
            <label>Message</label>
            <textarea class="form-control" name="message"
id="message"
                rows="4"></textarea>
        </div>
</div>
        <input type="submit" name="send" value="Submit"
class="btn btn-primary btn-block">
      </form>
   </div>
```

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ Empêcher les requêtes CSRF(csrf_token ())

Si vous inspectez la page après l'exécution, vous obtiendrez la même sortie de l'exemple précédent.

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ Empêcher les requêtes CSRF(csrf_field ())

csrf_field ()

Cette fonction crée un champ masqué pour le formulaire HTML où il est utilisé et génère un jeton CSRF.

✓ **Syntaxe**

```
<form method = "POST“ action="/profile">
  {{ csrf_field() }}
  .....
</form>
```

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ Empêcher les requêtes CSRF(csrf_field ())

✓ **Activité**

Considérez les lignes de code suivantes. Ils affichent un formulaire qui prend trois paramètres en entrée: Nom & prénom, email et message.

Développer en back-end

```
<div class="container mt-5">
        <form action="" method="post">
        {{ csrf_field() }}
<div class="row">
            <div class="form-group col-6 lg" >
                <label>Prénom & Nom</label>
                <input type="text" class="form-control"
name="prenomNom" id="prenomNom">
            </div>
            <div class="form-group col-6 lg">
                <label>Email</label>
                <input type="email" class="form-control" name="email"
                    id="email">
            </div>
</div>
```

Développer en back-end

```
<div class="row">
        <div class="form-group col-12 lg">
            <label>Message</label>
            <textarea class="form-control"
name="message" id="message"
                rows="4"></textarea>
        </div>
</div>
        <input type="submit" name="send"
value="Submit" class="btn btn-primary btn-block">
      </form>
   </div>
```

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ Exclure les URI de la protection CSRF

Parfois, vous souhaiterez peut-être exclure un ensemble d'URI de la protection CSRF.

En règle générale, vous devez placer ces types de routes en dehors du **web** groupe de middleware qui

App\Providers\RouteServiceProvider s'applique à toutes les routes du *routes/web.php* fichier.

Cependant, vous pouvez également exclure les routes en ajoutant leurs URI à la *$except* propriété du *VerifyCsrfTokenmiddleware*

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ Exclure les URI de la protection CSRF

```
namespace App\Http\Middleware;
use Illuminate\Foundation\Http\Middleware\VerifyCsrfToken as Middleware;

class VerifyCsrfToken extends Middleware
{
        protected $except = [

                'les routes  à exclues'
        ];
}
```

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ X-CSRF-TOKEN

En plus de vérifier le jeton CSRF en tant que paramètre POST, le *App\Http\Middleware\VerifyCsrfToken* middleware vérifiera également l' X-CSRF-TOKEN en-tête de la requête. Vous pouvez, par exemple, stocker le jeton dans une méta balise HTML :

```
<meta name="csrf-token" content="{{ csrf_token() }}">
```

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ X-CSRF-TOKEN

Ensuite, vous pouvez demander à une bibliothèque comme jQuery d'ajouter automatiquement le jeton à tous les en-têtes de requête. Cela fournit une protection CSRF simple et pratique pour vos applications basées sur AJAX utilisant la technologie JavaScript héritée :

```
$.ajaxSetup({
    headers: {
        'X-CSRF-TOKEN': $('meta[name="csrf-token"]').attr('content')
    }
});
```

Développer en back-end

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel

Protection CSRF(Cross-Site Request Forgery)

Falsification de requêtes intersites

✓ X-XSRF-TOKEN

Laravel stocke le jeton CSRF actuel dans un XSRF-TOKEN cookie crypté qui est inclus avec chaque réponse générée par le Framework. Vous pouvez utiliser la valeur du cookie pour définir l' X-XSRF-TOKEN en-tête de la demande.

Ce cookie est principalement envoyé pour la commodité des développeurs, car certains Framework et bibliothèques JavaScript, comme Angular et Axios, placent automatiquement sa valeur dans l'en- X-XSRF-TOKEN tête des requêtes de même origine.